

Shopify Developer Guide

A concise, practical reference for developers

Shopify Standard Plans vs Plus • APIs • Tokens & Scopes • Hydrogen

Core idea In a Hydrogen project, the Storefront API powers products, search, and cart. Use the Admin API for store management, the Customer Account API for signed-in customers, and Webhooks for events.

Prepared for

Remaa Yosef

Focus

Developer perspective

API version

2026-04 (stable in June 2026)

1. The Big Picture: How Shopify Works

Shopify manages the commerce backend: products, prices, inventory, customers, orders, checkout, and payments. You build the storefront or application that works with this data.



2. Shopify Standard Plans vs Shopify Plus

Important “Normal Shopify” is not an official plan name; it usually means a non-Plus plan. Plus is not a separate platform or API. It uses the same foundation with broader enterprise capabilities.

Developer view	Non-Plus plans	Shopify Plus
Core APIs	The same core APIs	The same core APIs
Hydrogen / Headless	Available	Available with greater scale and resources
Checkout	Customization on available surfaces	Deeper customization in Information, Shipping, and Payment
B2B	Features depend on the plan	Broader catalogs, payment rules, and workflows
Testing and expansion	Standard store management	Expansion stores and unlimited staging stores
API resources	Standard limits	Higher capacity and possible limit increases

When does Plus matter? When you need sensitive checkout customization, multiple stores and staging environments, advanced B2B workflows, or large integrations. Hydrogen by itself does not require Plus.

3. Shopify API Map

Start with one question: who is using this operation - the shopper, the signed-in customer, the merchant, or Shopify itself when an event occurs?

Storefront API Shopper-facing commerce: products, collections, search, cart, and checkout URL.	GraphQL Admin API Server-side store management: products, orders, inventory, and customers.
Customer Account API Signed-in customer data: profile, addresses, and order history.	Webhooks Notifications sent to your app when an order, product, or app event occurs.
Shopify Functions Backend logic for discounts, delivery, payment, and validation.	Checkout UI Extensions Safe UI inside checkout, such as a banner, checkbox, or field.

Quick Decision Guide

What you need	Best choice
Product, collection, search, or cart	Storefront API
Orders belonging to the current customer	Customer Account API
Inventory or all store orders	GraphQL Admin API
A reaction to an event	Webhook
Logic or UI inside checkout	Function / UI Extension

Most likely primary API for your project The Storefront API, because Hydrogen uses it to fetch commerce data and build the shopping experience.

4. Storefront API + Hydrogen

The Storefront API is a GraphQL API designed for shopper-facing experiences. It reads products, collections, and search results; creates and updates carts; and returns the checkout URL.

GraphQL in One Minute

Concept	Meaning
Query	Reads data.
Mutation	Creates or changes data.
Variables	Dynamic values passed into an operation.
Connection	A paginated list.
Handle	A URL-friendly identifier for a product or collection.

Example: Fetching a Product in Hydrogen

```
export async function loader({params, context}) {
  const {product} = await context.storefront.query(
    PRODUCT_QUERY,
    {variables: {handle: params.productHandle}}
  );
  return {product};
}

const PRODUCT_QUERY = `#graphql
  query Product($handle: String!) {
    product(handle: $handle) {
      id
      title
      variants(first: 10) {
        nodes { id title availableForSale price { amount currencyCode } }
      }
    }
  }
`;
```

Flow The route extracts the handle → the loader calls context.storefront.query() → Shopify returns only the requested fields → React renders the product.

5. Supporting APIs and Tools

GraphQL Admin API For a backend or admin app: update products and inventory, or read store orders. Keep the Admin token on the server.	Customer Account API For the current signed-in customer only: profile, addresses, and order history.
Webhooks Shopify sends an event such as orders/create to your endpoint instead of making your app poll repeatedly.	Shopify Functions Backend logic that runs during commerce flows: discounts, delivery, payment, and validation.

Function vs Checkout UI Extension

Tool	What it changes	Example
Function	Business logic and decisions	Apply a discount or block checkout
UI Extension	Interface shown to the customer	Display a banner or checkbox

Plus restriction Checkout UI extensions on the Information, Shipping, and Payment steps require Shopify Plus.

Three Webhook Rules

- Verify that the request truly came from Shopify before processing it.
- Make the handler idempotent, because the same event can be delivered more than once.
- Respond quickly and move heavy work to a queue when needed.

6. Tokens & Scopes

Direct answer No token starts with open access. The token type defines the family of permissions it can use, while scopes define what the application was actually granted.

Access type	Where it is used	Examples
Admin token	Server / Admin app	read_products, read_orders
Storefront token	Public in the browser or private on the server	unauthenticated_read_product_listings
Customer token	Signed-in customer session	Access to that customer's account data

Some basic Storefront API operations can work without a token, while protected capabilities require Storefront API access.

How Are Admin Scopes Defined?

Required and optional scopes are declared in the app configuration. The merchant approves them, and the issued token receives only the granted access.

```
[access_scopes]
scopes = "read_products,write_products,read_orders"
optional_scopes = ["write_discounts"]
```

- Missing scope = the query or mutation is rejected.
- Adding a required scope can require the merchant to approve access again.

Security rule Always follow least privilege. Keep sensitive tokens in server-side secrets, and inspect userErrors after mutations.

7. A Typical Hydrogen Project Flow

Page / operation	Tool	Result
Home	Storefront API	Featured products and collections
Collection	Storefront API	Products, filters, and pagination
Product page	Storefront API	Variants, price, and availability
Cart	Storefront cart mutations	Create, add, update, and remove
Checkout	cart.checkoutUrl	Redirect the customer to Shopify Checkout
My Account	Customer Account API	Profile, addresses, and orders

Short Cart Example

```
mutation CreateCart($input: CartInput!) {
  cartCreate(input: $input) {
    cart { id totalQuantity checkoutUrl }
    userErrors { field message }
  }
}
```

Variables:

```
{
  "input": {
    "lines": [{
      "merchandiseId": "gid://shopify/ProductVariant/123",
      "quantity": 2
    }]
  }
}
```

Watch this A cart normally receives a Product Variant ID, not a Product ID, because the customer adds a specific choice such as Black - Medium.

Checklist Identify whether the operation belongs to a shopper, merchant, customer, or event; pin an API version; use variables and pagination; then verify scopes and userErrors.

8. Summary and References

The Answers to Remember

Question	Answer
What is the main API with Hydrogen?	Storefront API.
How do I manage the store?	GraphQL Admin API from the server.
How do I show a customer's orders?	Customer Account API after sign-in.
How do I know an event occurred?	Webhook.
How do I customize checkout?	Functions for logic and UI Extensions for interface.
Is Shopify Plus a different API?	No. It uses the same foundation with additional enterprise capabilities and limits.
Does a token start fully open?	No. Its type and granted scopes define its permissions.

Official References

1. [Shopify APIs overview](#)
2. [Storefront API 2026-04](#)
3. [GraphQL Admin API 2026-04](#)
4. [API access scopes](#)
5. [Webhooks and Functions](#)
6. [Checkout UI extensions](#)
7. [Shopify Plus features](#)

Final mental model The Storefront API presents commerce, the Admin API manages it, the Customer Account API serves the signed-in customer, Webhooks report events, and Functions or Extensions customize the buying journey.